02 - Shared Preprocessing Setup

Milestone 2: Creating the common data contract and validation split used by the text and image preprocessing workflows.

1. Purpose

The Rakuten challenge provides labels for the training catalogue but not for `X_test_update.csv`. Local model evaluation therefore requires a held-out part of the labelled training data.

This notebook is the single owner of the shared training/validation assignment. It:

- loads and validates the labelled training catalogue;
- distinguishes the official test catalogue from local validation data;
- creates or reuses a stratified 80/20 split;
- stores the assignment under `data/processed/data_split.csv`;
- verifies coverage and class balance;
- defines the split contract consumed by the text and image notebooks.

Modality-specific cleaning and transformations do not belong in this notebook.

2. Imports and Configuration

```
Environment and shared data utilities loaded successfully.
```

3. Load and Validate the Challenge Tables

The original dataframe index is preserved as `row_id`. It is the common key used to join split assignments, text features, image metadata and model predictions.

	source	rows	target_available
0	labelled training catalogue	84916	True
1	official unlabelled test catalogue	13812	False

4. Dataset Roles

The official test catalogue cannot be used to calculate F1 scores because its labels are hidden. It is reserved for final blind predictions.

The local evaluation sets are therefore created only from the labelled catalogue:

```
Labelled catalogue
├── train       80%
└── validation  20%

Official test catalogue
└── final predictions only
```

5. Create or Load the Shared Split

The baseline split is stratified by `prdtypecode`, uses `random_state=42`, and assigns 20% of the labelled products to validation. Existing assignments are reused so repeated notebook runs cannot silently reshuffle the evaluation set.

This first baseline is stratified but not duplicate-aware. The audit found repeated titles and exact duplicate images, so a grouped split should later be evaluated as a leakage-sensitivity check. It should not silently replace the baseline because both validation strategies answer useful but different questions.

```
Loaded existing split manifest: /Users/wahajaslam/capstone_raukaten/data/processed/data_split.csv
```

6. Validate Coverage and Class Balance

A stale, incomplete or ambiguous split file must fail before it reaches either modality-specific workflow.

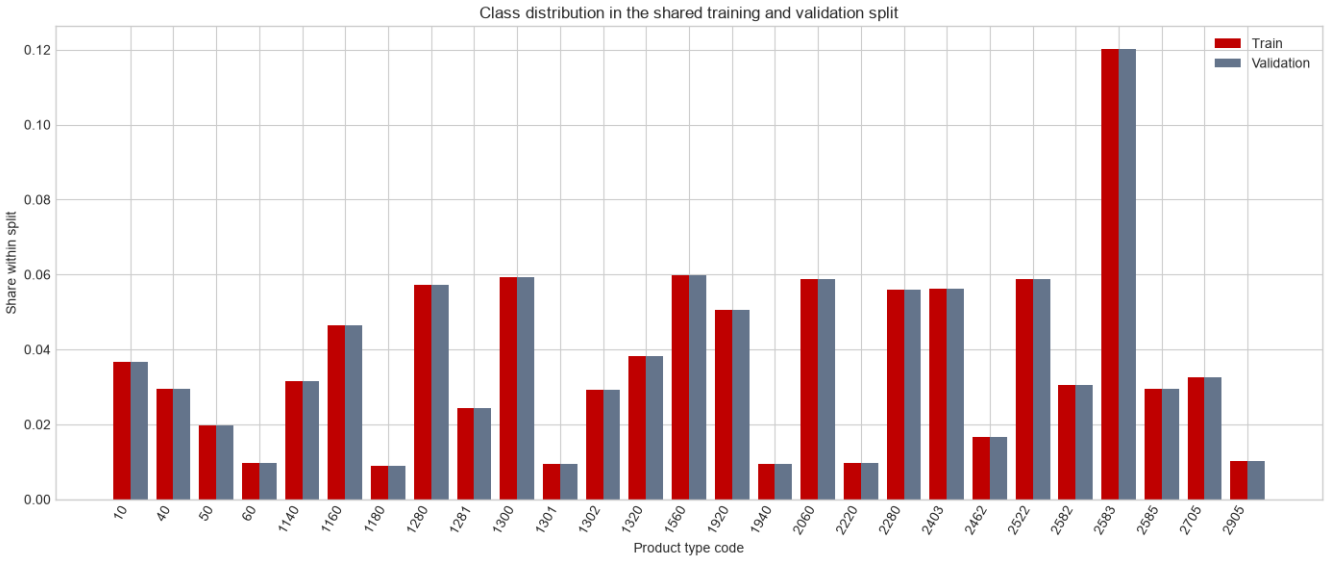
split	products	classes	share
train	67932	27	80.0%
validation	16984	27	20.0%

```
Maximum absolute class-share difference between train and validation: 0.003%
```

split prdtypecode	train	validation	absolute_difference
2905	1.027%	1.024%	0.003%
60	0.980%	0.977%	0.003%
1301	0.951%	0.948%	0.003%
1940	0.945%	0.948%	0.003%
40	2.953%	2.956%	0.003%
1160	4.655%	4.657%	0.003%
1920	5.067%	5.069%	0.003%
2060	5.879%	5.882%	0.003%
1560	5.974%	5.976%	0.003%
1320	3.817%	3.815%	0.002%

7. Visual Class-Balance Check

The chart compares class shares rather than raw counts because the training and validation partitions have different sizes. Closely aligned bars confirm that stratification preserved the target distribution.



8. Contract for Modality-Specific Notebooks

Both `02_preprocessing_image.ipynb` and the future `02_preprocessing_text.ipynb` must:

1. load `data/processed/data_split.csv`;
2. join it to the labelled catalogue through `row_id`;
3. validate the join as one-to-one;
4. train preprocessing or modelling components only on rows labelled `train`;
5. reserve rows labelled `validation` for evaluation;
6. keep the official test catalogue separate and unlabelled.

Because the entire `data/` directory is excluded from Git, team members who do not receive the manifest separately must run this notebook before running either modality-specific notebook.

9. Result and Next Step

The shared baseline split is now defined independently of text and image preprocessing. The modality notebooks can be developed in parallel while evaluating both classifiers on exactly the same held-out products.

The next steps are to implement text cleaning in `02_preprocessing_text.ipynb` and image transformations in `02_preprocessing_image.ipynb`. Model fitting remains part of the modelling milestone.